

50 Spring Boot Interview Questions

For Freshers & Internship Candidates

Covers Spring Core • Spring Boot • REST API • JPA/Hibernate • Security • Microservices

■ SECTION 1: Spring Core Fundamentals

Q1. What is the Spring Framework?

Spring is a lightweight, open-source Java framework for building enterprise applications. It provides features like Dependency Injection (DI), Aspect-Oriented Programming (AOP), and integration with various technologies. It simplifies Java EE development.

Q2. What is Dependency Injection (DI)?

DI is a design pattern where an object's dependencies are provided by an external source (the Spring container) rather than the object creating them itself. Types:

- Constructor Injection – Dependencies passed via constructor.
- Setter Injection – Dependencies passed via setter methods.
- Field Injection – Dependencies injected directly into fields using `@Autowired`.

Q3. What is Inversion of Control (IoC)?

IoC means the control of object creation and lifecycle is inverted from the application to the Spring IoC container. The container manages beans, their creation, wiring, and destruction.

Q4. What is a Spring Bean?

A Spring Bean is an object that is managed by the Spring IoC container. Beans are created, configured, and managed by Spring. They are defined using `@Component`, `@Service`, `@Repository`, `@Controller`, or in XML/Java config.

Q5. What are the different bean scopes in Spring?

- Singleton (default) – One instance per Spring container.
- Prototype – New instance every time the bean is requested.
- Request – One instance per HTTP request (web apps).
- Session – One instance per HTTP session.
- Application – One instance per ServletContext.

Q6. What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

All are specializations of `@Component` (stereotype annotations):

- `@Component` – Generic Spring-managed component.
- `@Service` – Business logic layer.
- `@Repository` – Data access layer; adds exception translation.
- `@Controller` – Presentation layer; handles HTTP requests.

Q7. What is `@Autowired`?

`@Autowired` tells Spring to inject a dependency automatically by type. Can be used on constructors, setters, or fields. If multiple beans of the same type exist, use `@Qualifier` to specify which one.

Q8. What is the difference between `@Autowired` and `@Inject`?

Both perform dependency injection. @Autowired is Spring-specific. @Inject is from the Java CDI standard (javax.inject). @Autowired has a required attribute (default true); @Inject does not.

■ SECTION 2: Spring Boot Basics

Q9. What is Spring Boot? How is it different from Spring?

Spring Boot is an opinionated framework built on top of Spring that simplifies configuration and setup. Key differences:

- Spring requires manual configuration; Spring Boot uses auto-configuration.
- Spring Boot has embedded servers (Tomcat, Jetty); no need to deploy WAR files.
- Spring Boot provides starter dependencies (spring-boot-starter-web, etc.).

Q10. What is auto-configuration in Spring Boot?

Auto-configuration automatically configures Spring application based on the dependencies present in the classpath. For example, if spring-boot-starter-web is on classpath, Spring Boot auto-configures DispatcherServlet, Jackson, etc. Enabled by @EnableAutoConfiguration (included in @SpringBootApplication).

Q11. What does @SpringBootApplication do?

@SpringBootApplication is a convenience annotation that combines:

- @Configuration – Marks as a configuration class.
- @EnableAutoConfiguration – Enables auto-configuration.
- @ComponentScan – Scans components in the current package and sub-packages.

Q12. What are Spring Boot Starters?

Starter are pre-configured dependency descriptors that simplify Maven/Gradle configuration. Examples:

- spring-boot-starter-web – For building web/REST applications.
- spring-boot-starter-data-jpa – For JPA/Hibernate.
- spring-boot-starter-security – For Spring Security.
- spring-boot-starter-test – For testing.

Q13. What is application.properties / application.yml?

These are configuration files in Spring Boot used to configure application settings such as server port, database URL, logging levels, etc. YAML (.yml) is more readable for nested configurations.

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost/mydb
spring.jpa.hibernate.ddl-auto=update
```

Q14. What is Spring Boot Actuator?

Actuator provides production-ready features for monitoring and managing Spring Boot applications. It exposes endpoints like /actuator/health, /actuator/info, /actuator/metrics, /actuator/beans for operational insight.

Q15. What is the difference between @RestController and @Controller?

- @Controller – Returns view names (used in MVC). Must use @ResponseBody on each method to return data.
- @RestController – Combination of @Controller + @ResponseBody. Every method returns data (JSON/XML) directly to the response body.

Q16. What is CommandLineRunner and ApplicationRunner?

Both are interfaces used to execute code after the Spring Boot application starts.

- CommandLineRunner – run(String... args) receives raw string arguments.
- ApplicationRunner – run(ApplicationArguments args) provides more structured argument access.

SECTION 3: REST API Development

Q17. What is a REST API?

REST (Representational State Transfer) is an architectural style for designing networked applications. It uses HTTP methods: GET (read), POST (create), PUT (update), DELETE (delete), PATCH (partial update). REST APIs are stateless and use JSON/XML for data exchange.

Q18. What are the key annotations for building REST APIs in Spring Boot?

- @RestController – Marks class as REST controller.
- @RequestMapping – Maps URL to class/method.
- @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping – HTTP method-specific mappings.
- @PathVariable – Extracts value from URL path.
- @RequestParam – Extracts query parameters.
- @RequestBody – Binds HTTP request body to an object.

Q19. What is ResponseEntity?

ResponseEntity represents the full HTTP response including status code, headers, and body. It gives full control over the response.

```
return ResponseEntity.status(HttpStatus.CREATED).body(savedUser);
```

Q20. What is the difference between @PathVariable and @RequestParam?

- @PathVariable – Extracts a value from the URI path. Example: /users/{id}
- @RequestParam – Extracts a query parameter from the URL. Example: /users?id=1

Q21. How do you handle exceptions globally in Spring Boot?

Using @ControllerAdvice (or @RestControllerAdvice) with @ExceptionHandler methods. This centralizes exception handling across all controllers instead of handling in each controller separately.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity handleNotFound(Exception e) {
        return ResponseEntity.status(404).body(e.getMessage());
    }
}
```

Q22. What is CORS and how to enable it in Spring Boot?

CORS (Cross-Origin Resource Sharing) allows or restricts web applications running on one origin to make requests to another origin. Enable globally using WebMvcConfigurer or per-method using @CrossOrigin annotation.

Q23. What are HTTP status codes commonly used in REST APIs?

- 200 OK – Successful GET/PUT.
- 201 Created – Successful POST.

- 204 No Content – Successful DELETE.
- 400 Bad Request – Invalid input.
- 401 Unauthorized – Authentication required.
- 403 Forbidden – Access denied.
- 404 Not Found – Resource not found.
- 500 Internal Server Error – Server-side error.

■ SECTION 4: Spring Data JPA & Hibernate

Q24. What is Spring Data JPA?

Spring Data JPA simplifies database access by providing repository interfaces. It reduces boilerplate CRUD code. Simply extend `JpaRepository` and Spring auto-generates the implementation.

Q25. What is the difference between JPA and Hibernate?

- JPA (Java Persistence API) – A specification/interface for ORM in Java.
- Hibernate – A popular implementation of the JPA specification. Spring Boot uses Hibernate as the default JPA provider.

Q26. What are the JPA annotations for entity mapping?

- `@Entity` – Marks class as a database table.
- `@Table` – Specifies table name.
- `@Id` – Primary key.
- `@GeneratedValue` – Auto-generates ID.
- `@Column` – Maps field to column.
- `@OneToMany`, `@ManyToOne`, `@ManyToMany`, `@OneToOne` – Relationships.

Q27. What is the difference between `save()` and `saveAndFlush()`?

- `save()` – Saves entity; may batch with other operations; flushes at transaction end.
- `saveAndFlush()` – Saves entity and immediately flushes changes to the database within the current transaction.

Q28. What is JPQL?

JPQL (Java Persistence Query Language) is an object-oriented query language similar to SQL but operates on JPA entities rather than database tables. Used with `@Query` annotation in Spring Data JPA.

```
@Query("SELECT u FROM User u WHERE u.email = :email")
User findByEmail(@Param("email") String email);
```

Q29. What is the difference between `FetchType.LAZY` and `FetchType.EAGER`?

- `EAGER` – Related entities are loaded immediately with the parent entity.
- `LAZY` – Related entities are loaded only when accessed. `LAZY` is preferred for performance; `EAGER` can cause N+1 query problems.

Q30. What is the N+1 query problem?

When fetching a list of N entities, if each entity lazily loads a related entity, it results in 1 (parent query) + N (child queries) = N+1 queries total. Solved using `JOIN FETCH` in JPQL or `@EntityGraph`.

Q31. What are the different cascade types in JPA?

CascadeType defines which operations on parent should cascade to child:

- ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH.

Example: @OneToMany(cascade = CascadeType.ALL) means all operations on parent cascade to children.

Q32. What is @Transactional?

@Transactional ensures that a method or class runs within a database transaction. If the method completes successfully, the transaction commits. If an exception occurs, it rolls back. Applied at service layer.

■ SECTION 5: Spring Security

Q33. What is Spring Security?

Spring Security is a powerful authentication and authorization framework for Java applications. It handles login, logout, password encoding, role-based access control, JWT, OAuth2, and CSRF protection.

Q34. What is the difference between Authentication and Authorization?

- Authentication – Verifying who the user is (login with username/password).
- Authorization – Verifying what the user is allowed to do (role-based access).

Q35. What is JWT (JSON Web Token)?

JWT is a compact, URL-safe token used for authentication. It consists of three parts: Header (algorithm), Payload (claims/user data), Signature. After login, the server issues a JWT; the client sends it in the Authorization header for subsequent requests.

Q36. What is BCryptPasswordEncoder?

BCryptPasswordEncoder is used to hash passwords before storing in the database. BCrypt is a one-way hash function; the original password cannot be recovered. Spring Security recommends BCrypt for password encoding.

```
PasswordEncoder encoder = new BCryptPasswordEncoder();  
String hashed = encoder.encode("myPassword");
```

Q37. What is CSRF and how does Spring Security handle it?

CSRF (Cross-Site Request Forgery) is an attack where unauthorized commands are sent from a trusted user. Spring Security enables CSRF protection by default for web apps. For REST APIs using JWT, CSRF is typically disabled (csrf().disable()).

■ SECTION 6: Microservices & Advanced Topics

Q38. What are Microservices?

Microservices is an architectural style where an application is built as a collection of small, independent services, each running in its own process and communicating via APIs. Benefits: independent deployment, scalability, fault isolation.

Q39. What is Spring Cloud?

Spring Cloud provides tools for building distributed systems and microservices. Key components:

- Eureka – Service discovery.

- Ribbon/LoadBalancer – Client-side load balancing.
- Feign – Declarative REST client.
- Config Server – Centralized configuration.
- API Gateway – Single entry point.

Q40. What is the difference between RestTemplate and WebClient?

- RestTemplate – Synchronous, blocking HTTP client. Being deprecated in newer Spring versions.
- WebClient – Non-blocking, reactive HTTP client introduced in Spring 5. Recommended for new applications.

Q41. What is an API Gateway?

An API Gateway is a single entry point for all client requests in a microservices architecture. It handles routing, authentication, rate limiting, load balancing, and logging. Spring Cloud Gateway is the modern choice.

Q42. What is service discovery? What is Eureka?

Service discovery allows microservices to find each other dynamically without hardcoded URLs. Netflix Eureka is a service registry where services register themselves; other services query Eureka to find them.

Q43. What is the Circuit Breaker pattern?

Circuit Breaker prevents cascading failures in microservices. If a service fails repeatedly, the circuit 'opens' and fails fast without calling the service, giving it time to recover. Implemented using Resilience4j or Hystrix in Spring.

Q44. What is Spring Boot DevTools?

Spring Boot DevTools provides features for faster development:

- Automatic application restart on code changes.
- LiveReload support for browser refresh.
- Disabled caching in development mode.

Added via spring-boot-devtools dependency.

Q45. What is the difference between @Bean and @Component?

- @Component – Used on classes; Spring auto-detects via component scanning.
- @Bean – Used on methods inside @Configuration classes; manually declares a bean. Use @Bean when you need to configure a third-party class you don't own.

SECTION 7: Testing & Miscellaneous

Q46. How do you test a Spring Boot application?

- @SpringBootTest – Loads full application context for integration tests.
- @WebMvcTest – Tests only the web layer (controllers).
- @DataJpaTest – Tests only JPA repositories.
- MockMvc – Simulates HTTP requests without a real server.
- Mockito – Mocks dependencies for unit testing.

Q47. What is the difference between unit testing and integration testing?

- Unit Testing – Tests individual components in isolation (mocking dependencies). Fast.
- Integration Testing – Tests multiple components working together (real database, full context). Slower but more realistic.

Q48. What is Lombok and why is it used?

Lombok is a Java library that reduces boilerplate code using annotations:

- @Getter / @Setter – Auto-generates getters/setters.
- @NoArgsConstructor, @AllArgsConstructor – Generates constructors.
- @Data – Combines @Getter, @Setter, @ToString, @EqualsAndHashCode.
- @Builder – Implements Builder pattern.

Q49. What is the difference between @SpringBootTest and @WebMvcTest?

- @SpringBootTest – Loads the complete application context; used for full integration tests; slower.
- @WebMvcTest – Loads only the web layer (controllers, filters); faster; other beans must be mocked with @MockBean.

Q50. What is Swagger / OpenAPI in Spring Boot?

Swagger (OpenAPI) is a tool for documenting REST APIs. In Spring Boot, add springdoc-openapi-ui dependency and access auto-generated API docs at /swagger-ui.html. Supports @Operation, @ApiResponse, @Parameter annotations for detailed documentation.

■ Spring Boot Interview Tips

- ✓ Always understand the difference between Spring and Spring Boot — it's asked in almost every interview.
- ✓ Be ready to explain the flow of a REST API request from Controller → Service → Repository.
- ✓ Know how @Transactional works and when to use it.
- ✓ Practice building a small CRUD REST API with Spring Boot + JPA from scratch.
- ✓ Understand JPA relationships (OneToMany,ManyToOne) and their fetch types.
- ✓ Know at least basics of Spring Security — authentication vs authorization.
- ✓ Be familiar with Microservices concepts even for fresher roles — it shows awareness.

Best of luck with your Spring Boot interviews! ■